

Отчёт по лабораторной работе № 14

Расширенное программирование в shell

true

Цель работы

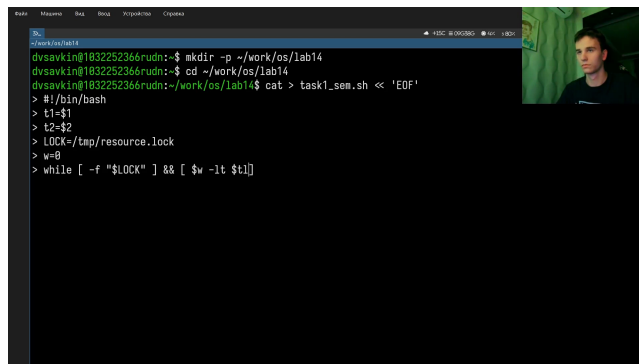
Написание сложных командных файлов с логическими конструкциями и циклами.

Задание

- Написать упрощённый семафор для синхронизации процессов.
- Реализовать аналог `map` командным файлом.
- Сгенерировать случайную последовательность латинских букв через `$RANDOM`.

Выполнение лабораторной работы

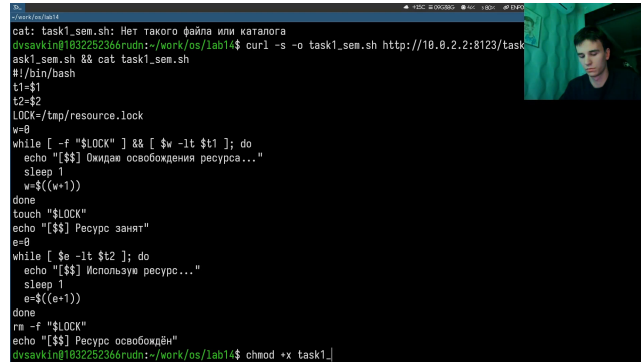
Создаём рабочий каталог и пишем скрипт семафора через `heredoc`: `mkdir -p ~/work/os/lab14, cat > task1_sem.sh << 'EOF'`. Скрипт принимает время ожидания `t1` и время занятия ресурса `t2`, использует файл-блокировку `/tmp/resource.lock`.



```
dvavkin@1832252366rudn:~$ mkdir -p ~/work/os/lab14
dvavkin@1832252366rudn:~$ cd ~/work/os/lab14
dvavkin@1832252366rudn:~/work/os/lab14$ cat > task1_sem.sh << 'EOF'
> #!/bin/bash
> t1=$1
> t2=$2
> LOCK=/tmp/resource.lock
> w=0
> while [ -f "$LOCK" ] && [ $w -lt $t1 ]
```

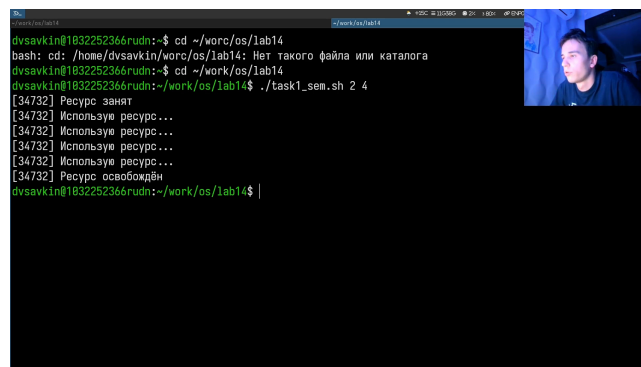
Полный текст `task1_sem.sh`: цикл ожидания освобождения ресурса (с сообщениями и счётчиком `w`), затем `touch "$LOCK"` и занятие ресурса на `t2`

секунд, в конце `rm -f "$LOCK"`.



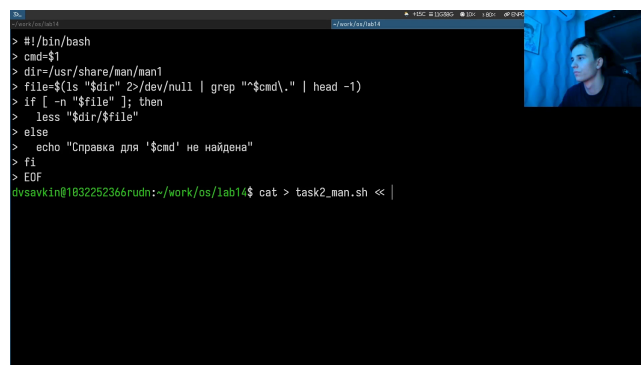
```
cat: task1_sem.sh: Нет такого файла или каталога
dysavkin@1832252366rudn:~/work/os/lab14$ curl -s -o task1_sem.sh http://10.0.2.2:8123/task1_sem.sh && cat task1_sem.sh
#!/bin/bash
t1=$1
t2=$2
LOCK=/tmp/resource.lock
w=0
while [ -f "$LOCK" ] && [ $w -lt $t1 ]; do
    echo "[$$] Ожидаю освобождения ресурса..."
    sleep 1
    w=$((w+1))
done
touch "$LOCK"
echo "[$$] Ресурс занят"
w=0
while [ $w -lt $t2 ]; do
    echo "[$$] Использую ресурс..."
    sleep 1
    w=$((w+1))
done
rm -f "$LOCK"
echo "[$$] Ресурс освобожден"
dysavkin@1832252366rudn:~/work/os/lab14$ chmod +x task1_
```

Даём права на выполнение и тестируем: `chmod +x task1_sem.sh, ./task1_sem.sh 2 4` — скрипт занимает ресурс и удерживает его 4 секунды с выводом PID (\$\$).



```
dysavkin@1832252366rudn:~$ cd ~/work/os/lab14
bash: cd: /home/dysavkin/work/os/lab14: Нет такого файла или каталога
dysavkin@1832252366rudn:~$ cd ~/work/os/lab14
dysavkin@1832252366rudn:~/work/os/lab14$ ./task1_sem.sh 2 4
[34732] Ресурс занят
[34732] Использую ресурс...
[34732] Использую ресурс...
[34732] Использую ресурс...
[34732] Использую ресурс...
[34732] Ресурс освобождён
dysavkin@1832252366rudn:~/work/os/lab14$ |
```

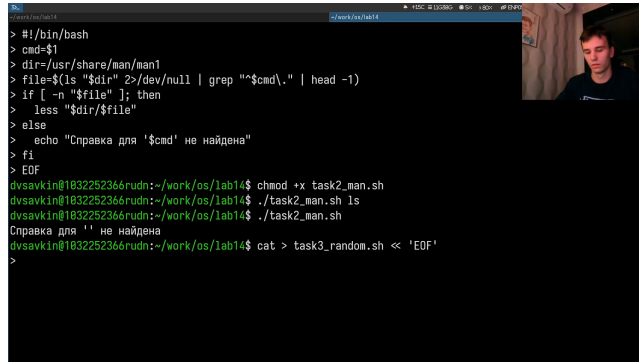
Пишем аналог `man`: `cat > task2_man.sh << 'EOF'`. Скрипт ищет файл справки по имени команды в `/usr/share/man/man1` через `ls | grep`, открывает через `less`, при отсутствии — выводит сообщение.



```
#!/bin/bash
> cmd=$1
> cmd=${1}
> dir=/usr/share/man/man1
> file=$(ls "$dir" 2>/dev/null | grep "^$cmd\." | head -1)
> if [ -n "$file" ]; then
>   less "$dir/$file"
> else
>   echo "Справка для '$cmd' не найдена"
> fi
> EOF
dysavkin@1832252366rudn:~/work/os/lab14$ cat > task2_man.sh << |
```

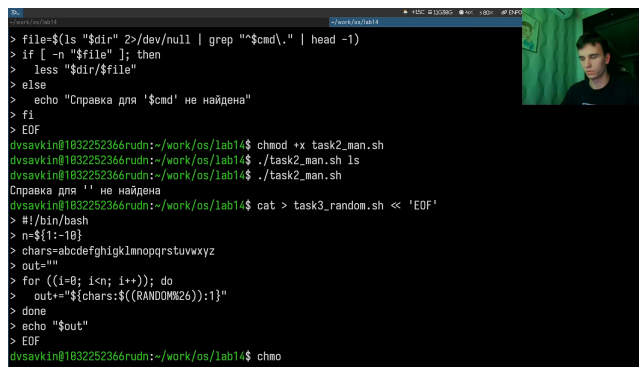
Тестируем: `chmod +x task2_man.sh, ./task2_man.sh ls` открывает

страницу справки, `./task2_man.sh` без аргумента выводит Справка для '' не найдена.



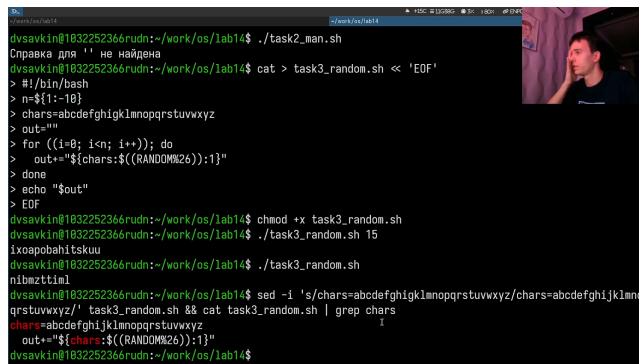
```
> #!/bin/bash
> cmd=$1
> dir=/usr/share/man/man1
> file=$(ls "$dir" 2>/dev/null | grep "^$cmd\" | head -1)
> if [ -n "$file" ]; then
>   less "$dir/$file"
> else
>   echo "Справка для '$cmd' не найдена"
> fi
> EOF
dysavkin@1032252366rudn:~/work/os/lab14$ chmod +x task2_man.sh
dysavkin@1032252366rudn:~/work/os/lab14$ ./task2_man.sh ls
dysavkin@1032252366rudn:~/work/os/lab14$ ./task2_man.sh
Справка для '' не найдена
dysavkin@1032252366rudn:~/work/os/lab14$ cat > task3_random.sh << 'EOF'
>
```

Пишем генератор случайных букв: `cat > task3_random.sh << 'EOF'`. Число букв берётся первым аргументом (по умолчанию 10), каждая буква — случайный символ из алфавита через `${chars:$( (RANDOM%26) ):1}`.



```
> file=$(ls "$dir" 2>/dev/null | grep "^$cmd\" | head -1)
> if [ -n "$file" ]; then
>   less "$dir/$file"
> else
>   echo "Справка для '$cmd' не найдена"
> fi
> EOF
dysavkin@1032252366rudn:~/work/os/lab14$ chmod +x task2_man.sh
dysavkin@1032252366rudn:~/work/os/lab14$ ./task2_man.sh ls
dysavkin@1032252366rudn:~/work/os/lab14$ ./task2_man.sh
Справка для '' не найдена
dysavkin@1032252366rudn:~/work/os/lab14$ cat > task3_random.sh << 'EOF'
> #!/bin/bash
> n=${1:-10}
> chars=abcdefghijklmnopqrstuvwxyz
> out=""
> for ((i=0; i<n; i++)); do
>   out+="${chars:$( (RANDOM%26) ):1}"
> done
> echo "$out"
> EOF
dysavkin@1032252366rudn:~/work/os/lab14$ chmo
```

Запускаем: `./task3_random.sh 15` и `./task3_random.sh`. Замечаем опечатку в алфавите (пропущена буква j) и исправляем на месте через `sed -i 's/.../.../' task3_random.sh`, проверяем результат через `grep chars`.



```
dysavkin@1032252366rudn:~/work/os/lab14$ ./task2_man.sh
Справка для '' не найдена
dysavkin@1032252366rudn:~/work/os/lab14$ cat > task3_random.sh << 'EOF'
> #!/bin/bash
> n=${1:-10}
> chars=abcdefghijklmnopqrstuvwxyz
> out=""
> for ((i=0; i<n; i++)); do
>   out+="${chars:$( (RANDOM%26) ):1}"
> done
> echo "$out"
> EOF
dysavkin@1032252366rudn:~/work/os/lab14$ chmod +x task3_random.sh
dysavkin@1032252366rudn:~/work/os/lab14$ ./task3_random.sh 15
ixooapohitkuu
dysavkin@1032252366rudn:~/work/os/lab14$ ./task3_random.sh
nibmztiml
dysavkin@1032252366rudn:~/work/os/lab14$ sed -i 's/.../.../' task3_random.sh
dysavkin@1032252366rudn:~/work/os/lab14$ cat task3_random.sh | grep chars
chars=abcdefghijklmnopqrstuvwxyz
out+="${chars:$( (RANDOM%26) ):1}"
dysavkin@1032252366rudn:~/work/os/lab14$
```

Выводы

Освоены командные файлы с циклами и условиями: упрощённый семафор на основе файла-блокировки для синхронизации процессов, обёртка над `map`, генератор случайных латинских букв через `$RANDOM`.

Ответы на контрольные вопросы

1. Найдите синтаксическую ошибку в `while [ $1 != "exit" ]`. Нет пробелов вокруг `[ и ]`: нужно `[ "$1" != "exit" ]`.
2. Как выполняется конкатенация строк в `bash`? Просто пишутся подряд: `c="$a$b"`.
3. Что делает утилита `seq` и чем её можно заменить? Выводит числа по порядку (`seq 1 10`); замена — `{1..10}` или `for ((i=1;i<=10;i++))`.
4. Чему равно `$ ( (10/3) )`? 3 — деление целочисленное.
5. Чем `zsh` отличается от `bash`? Более гибкие автодополнение и глобы, поддержка плагинов (`oh-my-zsh`), другой синтаксис массивов.
6. Корректен ли синтаксис `for ((a=1; a <= LIMIT; a++))`? Да, корректный C-подобный цикл.
7. Сравните `bash` с языками программирования (плюсы/минусы). Плюс — быстрая автоматизация системных задач без компиляции; минус — низкая производительность и плохая читаемость сложной логики.